Testing Dependencies with Overrides

Overriding dependencies during testing

There are some scenarios where you might want to override a dependency during testing.

You don't want the original dependency to run (nor any of the sub-dependencies it might have).

Instead, you want to provide a different dependency that will be used only during tests (possibly only some specific tests), and will provide a value that can be used where the value of the original dependency was used.

Use cases: external service

An example could be that you have an external authentication provider that you need to call.

You send it a token and it returns an authenticated user.

This provider might be charging you per request, and calling it might take some extra time than if you had a fixed mock user for tests.

You probably want to test the external provider once, but not necessarily call it for every test that runs.

In this case, you can override the dependency that calls that provider, and use a custom dependency that returns a mock user, only for your tests.

Use the `app.dependency_overrides` attribute

For these cases, your **FastAPI** application has an attribute `app.dependency_overrides`, it is a simple `dict`.

To override a dependency for testing, you put as a key the original dependency (a function), and as the value, your dependency override (another function).

And then **FastAPI** will call that override instead of the original dependency.

Python 3.10+

```python
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}


@app.get("/items/")
async def read_items(commons: Annotated[dict, Depends(common_parameters)]):
    return {"message": "Hello Items!", "params": commons}


@app.get("/users/")
async def read_users(commons: Annotated[dict, Depends(common_parameters)]):
    return {"message": "Hello Users!", "params": commons}
```

Ask AI

```python
client = TestClient(app)


async def override_dependency(q: str | None = None):
    return {"q": q, "skip": 5, "limit": 10}


app.dependency_overrides[common_parameters] = override_dependency


def test_override_in_items():
    response = client.get("/items/")
    assert response.status_code == 200
    assert response.json() == {
        "message": "Hello Items!",
        "params": {"q": None, "skip": 5, "limit": 10},
    }


def test_override_in_items_with_q():
    response = client.get("/items/?q=foo")
    assert response.status_code == 200
    assert response.json() == {
        "message": "Hello Items!",
        "params": {"q": "foo", "skip": 5, "limit": 10},
    }


def test_override_in_items_with_params():
    response = client.get("/items/?q=foo&skip=100&limit=200")
    assert response.status_code == 200
    assert response.json() == {
        "message": "Hello Items!",
        "params": {"q": "foo", "skip": 5, "limit": 10},
    }
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from fastapi import Depends, FastAPI
from fastapi.testclient import TestClient

app = FastAPI()


async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}


@app.get("/items/")
async def read_items(commons: dict = Depends(common_parameters)):
    return {"message": "Hello Items!", "params": commons}


@app.get("/users/")
async def read_users(commons: dict = Depends(common_parameters)):
    return {"message": "Hello Users!", "params": commons}


client = TestClient(app)


async def override_dependency(q: str | None = None):
    return {"q": q, "skip": 5, "limit": 10}


app.dependency_overrides[common_parameters] = override_dependency


def test_override_in_items():
    response = client.get("/items/")
    assert response.status_code == 200
    assert response.json() == {
        "message": "Hello Items!",
        "params": {"q": None, "skip": 5, "limit": 10},
    }


def test_override_in_items_with_q():
    response = client.get("/items/?q=foo")
    assert response.status_code == 200
    assert response.json() == {
        "message": "Hello Items!",
        "params": {"q": "foo", "skip": 5, "limit": 10},
    }


def test_override_in_items_with_params():
    response = client.get("/items/?q=foo&skip=100&limit=200")
    assert response.status_code == 200
    assert response.json() == {
        "message": "Hello Items!",
        "params": {"q": "foo", "skip": 5, "limit": 10},
    }
```

Tip

You can set a dependency override for a dependency used anywhere in your **FastAPI** application.

The original dependency could be used in a *path operation function*, a *path operation decorator* (when you don't use the return value), a `.include_router()` call, etc.

FastAPI will still be able to override it.

Then you can reset your overrides (remove them) by setting `app.dependency_overrides` to be an empty `dict`:

```
app.dependency_overrides = {}
```

Tip

If you want to override a dependency only during some tests, you can set the override at the beginning of the test (inside the test function) and reset it at the end (at the end of the test function).